UNIVERSIDAD AUTÓNOMA DE SINALOA

Facultad de Informática

REPORTE DE LABORATORIO 1

Protección y privacidad de datos

PostgreSQL · pgAdmin

Materia
Gestión de Seguridad y Desempeño de Base de Datos

Profesor
Alexis Moisés Montaño Araujo

Alumno
José Fernando Velázquez Espinoza

Fecha
2 de octubre de 2026

Evidencia 02: Enmascaramiento (masking)

Dato	Detalle
Estudiante	José Fernando Velázquez Espinoza
Actividad	Evidencia 02: Enmascaramiento (masking)
Materia	Gestión de Seguridad y Desempeño de Base de Datos
Motor / herramienta	PostgreSQL 16 · pgAdmin (Query Tool)
Script asociado	sql/02-enmascaramiento.sql
Fecha / contexto de ejecución	_____ (completar al ejecutar)

Objetivo

Proporcionar al área de soporte una vista con información parcialmente ocultada y evitar que el rol tenga acceso directo a la tabla original.

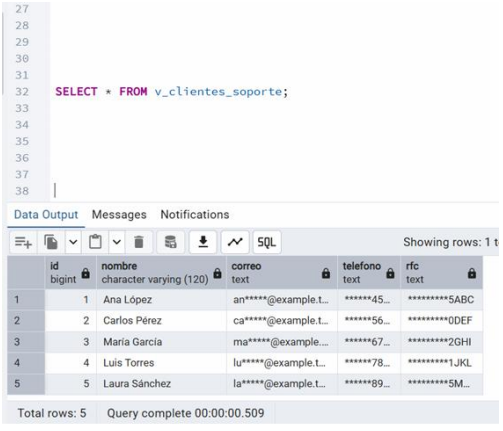
1. Creación del rol y de la vista

```
CREATE ROLE soporte NOLOGIN;

CREATE VIEW v_clientes_soporte AS
SELECT
    id,
    nombre,
    CONCAT(LEFT(correo, 2), '*****',
           SUBSTRING(correo FROM POSITION('@' IN correo))) AS correo,
    CONCAT('******', RIGHT(telefono, 4)) AS telefono,
    CONCAT('*********', RIGHT(rfc, 4)) AS rfc
FROM clientes;

SELECT * FROM v_clientes_soporte;
```

EVIDENCIA: Resultado de la consulta a la vista v_clientes_soporte



2. Permisos aplicados

```
GRANT USAGE ON SCHEMA public TO soporte;
GRANT SELECT ON v_clientes_soporte TO soporte;
REVOKE ALL ON clientes FROM soporte;

SELECT grantee, table_name, privilege_type
FROM information_schema.role_table_grants
WHERE grantee = 'soporte'
ORDER BY table_name, privilege_type;
```

EVIDENCIA: Permisos aplicados al rol soporte

Query | Query History

```
 6
 7  GRANT USAGE
 8  ON SCHEMA public;
 9  TO soporte_jose;
10
11  GRANT SELECT
12  ON v_clientes_soporte;
13  TO soporte_jose;
14
15  REVOKE ALL
16  ON clientes
17  FROM soporte_jose;
```

Data Output | Messages | Notifications

```
REVOKE

Query returned successfully in 545 msec.
```

3. Verificación como rol soporte

El rol es NOLOGIN, por lo que se usa SET ROLE desde la sesión del administrador.

```
SET ROLE soporte;

SELECT * FROM v_clientes_soporte;   -- permitido
SELECT * FROM clientes;             -- debe fallar

RESET ROLE;
```

EVIDENCIA: Consulta permitida a v_clientes_soporte (rol soporte)

```
SELECT * FROM v_clientes_soporte;
```

Data Output | Messages | Notifications

Showing rows: 1

	id bigint	nombre character varying (120)	correo text	telefono text	rfc text
1	1	Ana López	an*****@example.t...	******45...	*********5ABC
2	2	Carlos Pérez	ca*****@example.t...	******56...	*********0DEF
3	3	María García	ma*****@example....	******67...	*********2GHI
4	4	Luis Torres	lu*****@example.t...	******78...	*********1JKL
5	5	Laura Sánchez	la*****@example.t...	******89...	*********

Total rows: 5 | Query complete 00:00:00.346

EVIDENCIA: Intento rechazado sobre clientes, con el mensaje de error (permission denied for table clientes)

Query | Query History

```
12  SELECT current_user;
13
14
15  SELECT *
16  FROM v_clientes_soporte;
17  SELECT *
18  FROM clientes;
```

Data Output | Messages | Notifications

```
ERROR:  permission denied for table clientes

SQL state: 42501
```

4. Preguntas

¿Qué información continúa existiendo en la tabla original?

Todos los valores completos de correo, teléfono y RFC, además del resto de columnas. El enmascaramiento solo cambia la presentación; el dato almacenado no se modifica.

¿Qué información puede visualizar el usuario soporte?
id y nombre completos; el correo parcial (por ejemplo an*****@example.test); el teléfono con solo los últimos cuatro dígitos; y el RFC con solo los últimos cuatro caracteres. No ve edad, ciudad, código postal ni fecha de alta.

¿Por qué no es suficiente ocultar los datos únicamente en la aplicación?
Porque quien tenga credenciales de base de datos puede conectarse directamente (psql, pgAdmin, otra aplicación o un script) y evadir la aplicación. La protección debe aplicarse en la base de datos, donde realmente se controla el acceso.

¿Qué ocurriría si soporte tuviera acceso directo a clientes?
Podría consultar los valores completos de correo, teléfono y RFC y todas las demás columnas, anulando el enmascaramiento y violando el principio de mínimo privilegio.

5. Explicación del mecanismo utilizado

Se aplicó enmascaramiento dinámico mediante una vista (VIEW), combinado con GRANT y REVOKE. La vista presenta solo las columnas necesarias con los datos parcialmente ocultos y se ejecuta con los privilegios de su propietario, por lo que el rol soporte puede consultarla sin tener acceso a la tabla base. Se otorgó SELECT únicamente sobre la vista y se revocaron los privilegios sobre clientes (mínimo privilegio).